Day 20: Singleton & Static Peripheral Managers

Platform: nRF52840 | **RTOS:** Zephyr RTOS

Learning Objectives

Singleton Design Pattern

A **Singleton** ensures only one instance of a class exists. For hardware peripherals (UART, SPI bus), this makes sense — there's only one UART0 in the hardware:

```
class UartManager {
public:
    static UartManager &instance() {
        static UartManager inst;  // Meyers Singleton — thread-safe in C++11
        return inst;
    }
    // ...
private:
    UartManager() = default;                          // Private constructor
    UartManager(const UartManager &) = delete;        // No copy
    UartManager &operator=(const UartManager &) = delete;
};
// Usage:
UartManager::instance().send("hello");
```

Monostate Pattern

The **Monostate** pattern has all state as `static` members — every instance shares the same state. Unlike Singleton, multiple Monostate objects can be created (useful when you need default-constructible types but shared state):

```
class SystemLog {
public:
    static void log(const char *msg) { /* ... */ }
```

```
    static uint32_t error_count()    { return errors_; }
private:
    static uint32_t errors_;
};
uint32_t SystemLog::errors_ = 0;
```

K_MUTEX_DEFINE for C++ Singletons

Zephyr mutex macros must be used at file scope (they expand to static initialization). In a C++ class, use `k_mutex_init()` in the constructor:

```
class SafeManager {
    struct k_mutex mutex_;
public:
    SafeManager() { k_mutex_init(&mutex_); }
    void do_work() {
        k_mutex_lock(&mutex_, K_FOREVER);
        /* ... */
        k_mutex_unlock(&mutex_);
    }
};
```

Static Peripheral Manager

A static peripheral manager encapsulates all access to a peripheral:

```
class SpiManager {
public:
    static SpiManager &instance() { static SpiManager inst; return inst; }
    bool transfer(const uint8_t *tx, uint8_t *rx, size_t len);
private:
    const struct device *spi_dev_;
    struct k_mutex lock_;
};
```

All SPI access goes through `SpiManager::instance().transfer()` — thread-safe, no accidental concurrent access.

Thread-safe Singleton in Zephyr

Zephyr runs multiple threads — the Singleton must be thread-safe. In C++11, the **Meyers Singleton** (local `static`) is guaranteed thread-safe by the standard. However, verify that your compiler/linker supports this (`CONFIG_CPP=y` in Zephyr).

Global Object Initialization Order (Static Init Order Fiasco)

When you have multiple global objects in different `.cpp` files, the initialization order between files is **undefined**. If ObjectA's constructor uses ObjectB (defined in another file), ObjectB may not be initialized yet. Solution:

- Use Meyers Singleton (initialized on first use, not at startup)
- Use `init()` functions called from `main()` in the right order
- Avoid global objects with complex constructors

Practice Tasks

1. Add a call counter to your Singleton — verify it's the same instance across multiple calls
2. Implement a `FlashManager` Singleton that provides thread-safe Flash read/write operations
3. Demonstrate the static init order fiasco by having two global objects that depend on each other — then fix it with Meyers Singleton
4. Compare Singleton vs dependency injection for a sensor class — discuss testability trade-offs

---

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day20
west build -b nrf52840dk/nrf52840 .
west flash
```

```
# View output via RTT or UART serial (115200 baud)
west attach   # RTT viewer (requires J-Link)
```

Resources

- Zephyr Docs
- nRF52840 Product Specification
- Nordic DevZone
- Source: src/day20/main.cpp